

Threat Hunting con YARA

Individuazione di un impianto malevolo tramite regole di detection statica

Autore: Francesco Terracciano

Data: 23 agosto 2026

1. Obiettivo

L'esercitazione ha due finalità progressive. La prima consiste nel prendere confidenza con la sintassi di YARA scrivendo una regola elementare e verificandone il funzionamento su un file di prova creato appositamente. La seconda, di taglio operativo, simula una vera attività di threat hunting: un gruppo APT ha distribuito un impianto che ha modificato il proprio hash e assunto il nome di un file di sistema legittimo, rendendo inefficace qualsiasi ricerca per nome o per firma crittografica. La detection deve quindi basarsi su indicatori di compromissione (IoC) forniti da un report di Threat Intelligence: una stringa offuscata presente nel codice e una sequenza di byte usata per la memory injection.

2. Ambiente di lavoro e scelta della piattaforma

Erano disponibili due macchine virtuali: Kali Linux e FLARE VM. L'analisi è stata condotta interamente su Kali Linux (architettura arm64, virtualizzata con UTM su host macOS), con YARA 4.5.8-1 installato dai repository ufficiali. Di seguito le motivazioni della scelta.

2.1 L'architettura dell'host è irrilevante per l'analisi statica

YARA non esegue e non emula il campione: si limita a leggere il file e a confrontarne i byte con i pattern definiti nelle regole. Di conseguenza un binario PE compilato per Windows x86-64 può essere analizzato senza alcuna limitazione da una Kali su architettura arm64. Questa è la premessa che rende legittima l'intera scelta metodologica.

2.2 Isolamento e riduzione del rischio

Analizzare un campione Windows da un sistema Linux elimina in radice il rischio di esecuzione accidentale: anche in caso di doppio click o di errore nel comando, un PE non può essere eseguito nativamente sull'host di analisi. Trattandosi di un'attività puramente statica, non vi era alcuna necessità di eseguire il codice, e la piattaforma che rende l'esecuzione impossibile è preferibile a quella che la rende soltanto sconsigliata.

2.3 Toolchain nativa e verificabilità

Kali mette a disposizione, senza installazioni aggiuntive, tutti gli strumenti di triage necessari a corredare la scansione: sha256sum per l'identificazione univoca del campione, file per il riconoscimento del formato, strings e xxd per l'ispezione del contenuto. YARA stesso si installa con un singolo comando da repository, con versione tracciabile e riproducibile.

2.4 Adeguatezza allo scopo dimostrativo

Poiché l'obiettivo era documentare il procedimento, l'interfaccia a riga di comando risulta la soluzione più efficace: ogni passaggio corrisponde a un comando testuale esplicito, copiabile,

ripetibile e verificabile da terzi, e l'output è immediatamente leggibile in una relazione. Una toolchain grafica avrebbe richiesto una sequenza di screenshot per illustrare operazioni che qui occupano una riga.

FLARE VM resta la piattaforma d'elezione per le fasi successive di un'indagine reale — analisi dinamica in sandbox, debugging, ispezione approfondita dell'header PE con strumenti come PESTudio o CFF Explorer — ma per una detection basata su pattern matching non avrebbe aggiunto nulla, a fronte di una superficie di rischio maggiore.

3. Esercizio 1 — Prima scansione

3.1 Preparazione del bersaglio e dell'ambiente

È stato creato sul Desktop il file di prova config.txt contenente la stringa da rilevare, quindi installato YARA:

```
cd ~/Desktop
echo -n "Connessione_C2_Rilevata" > config.txt
sudo apt install yara
```

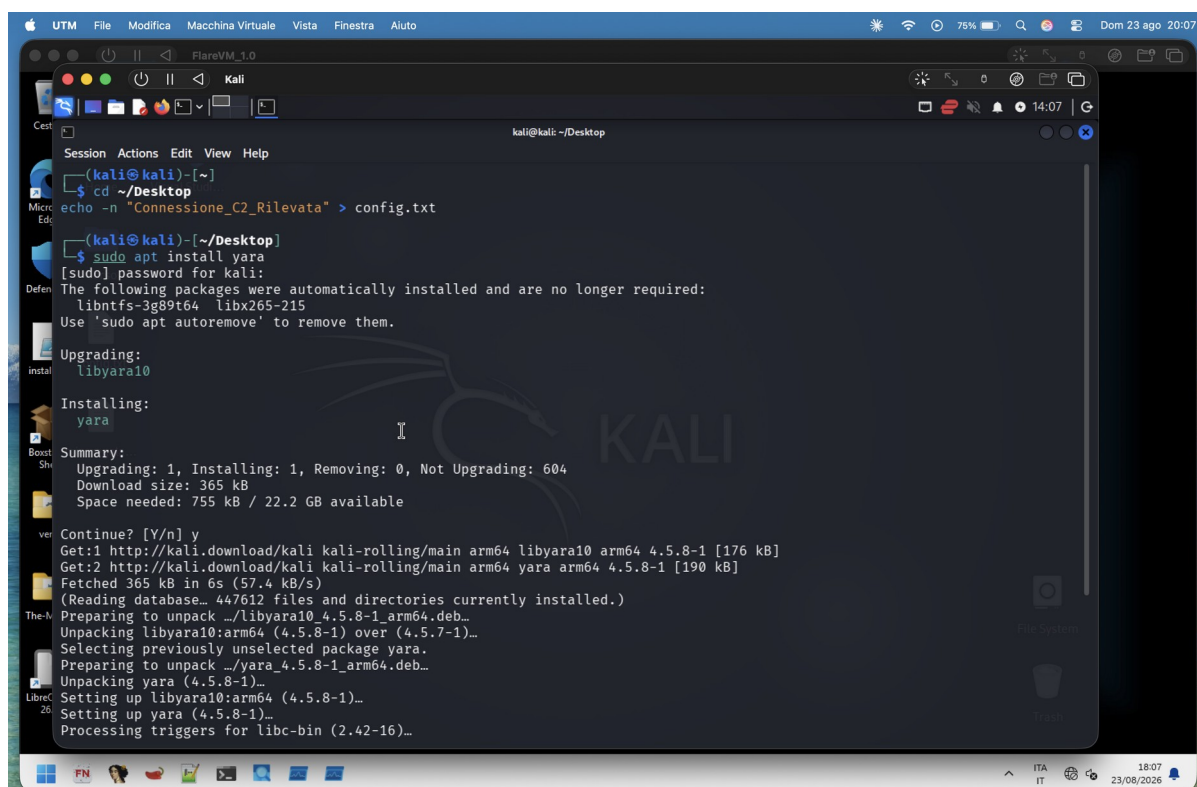


Fig. 1 — Creazione del file bersaglio e installazione di YARA 4.5.8-1 (con libyara10) dai repository Kali rolling.

3.2 Stesura della regola ed esecuzione

La regola dichiara una singola stringa con i modificatori ascii, wide e nocase, in modo da intercettarla sia in codifica a byte singolo sia in UTF-16LE e indipendentemente da maiuscole e minuscole. La condizione si limita alla presenza della stringa.

```
rule Rileva_Finto_Malware
```

```
{
  strings:
    $stringa_segreta = "Connessione_C2_Rilevata" ascii wide nocase

  condition:
    $stringa_segreta
}
```

Esecuzione della scansione:

```
yara mia_regola.yar config.txt
```

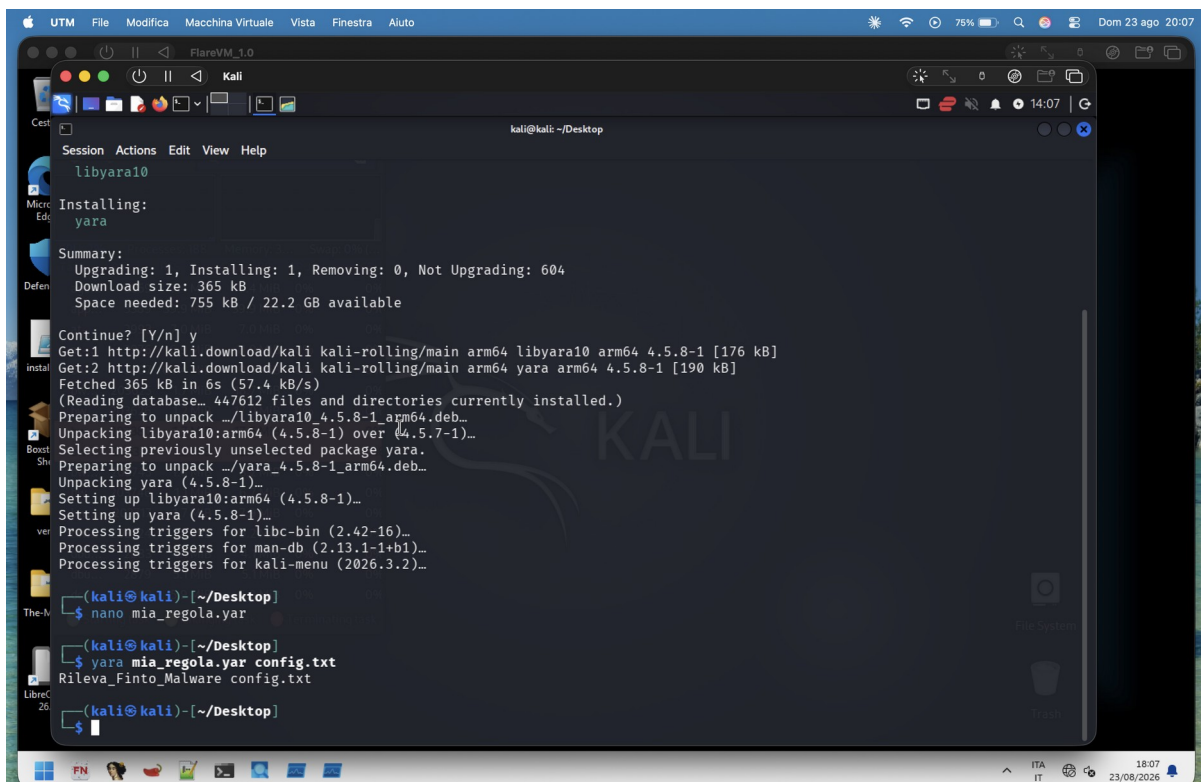


Fig. 2 — Creazione della regola con nano e match positivo: YARA restituisce il nome della regola seguito dal file che l'ha attivata.

L'output ottenuto — Rileva_Finto_Malware config.txt — conferma il corretto funzionamento della regola.

4. Esercizio 2 — La caccia al file

4.1 La regola di detection

La regola combina i due indicatori forniti dalla Threat Intelligence in una condizione OR: l'allarme scatta se il file contiene la stringa offuscata oppure la sequenza esadecimale di memory injection. La sezione meta documenta autore, descrizione e data, informazione essenziale quando le regole vengono condivise all'interno di un team.

```
rule caccia_malware
```

```

{
  meta:
    author = "Francesco Terracciano"
    description = "Impianto APT rinominato come processo di sistema Windows"
    date = "2026-08-23"

  strings:
    $stringa_offuscata = "AntiAV_Bypass_2026" ascii wide nocase
    $bytes_injection = { E2 9A 12 FF }

  condition:
    $stringa_offuscata or $bytes_injection
}

```

Due accortezze sintattiche: l'identificatore della regola non può contenere il punto (caccia_malware è il nome della regola, caccia_malware.yar quello del file) e la sequenza esadecimale va racchiusa tra parentesi graffe senza virgolette, altrimenti verrebbe interpretata come stringa letterale.

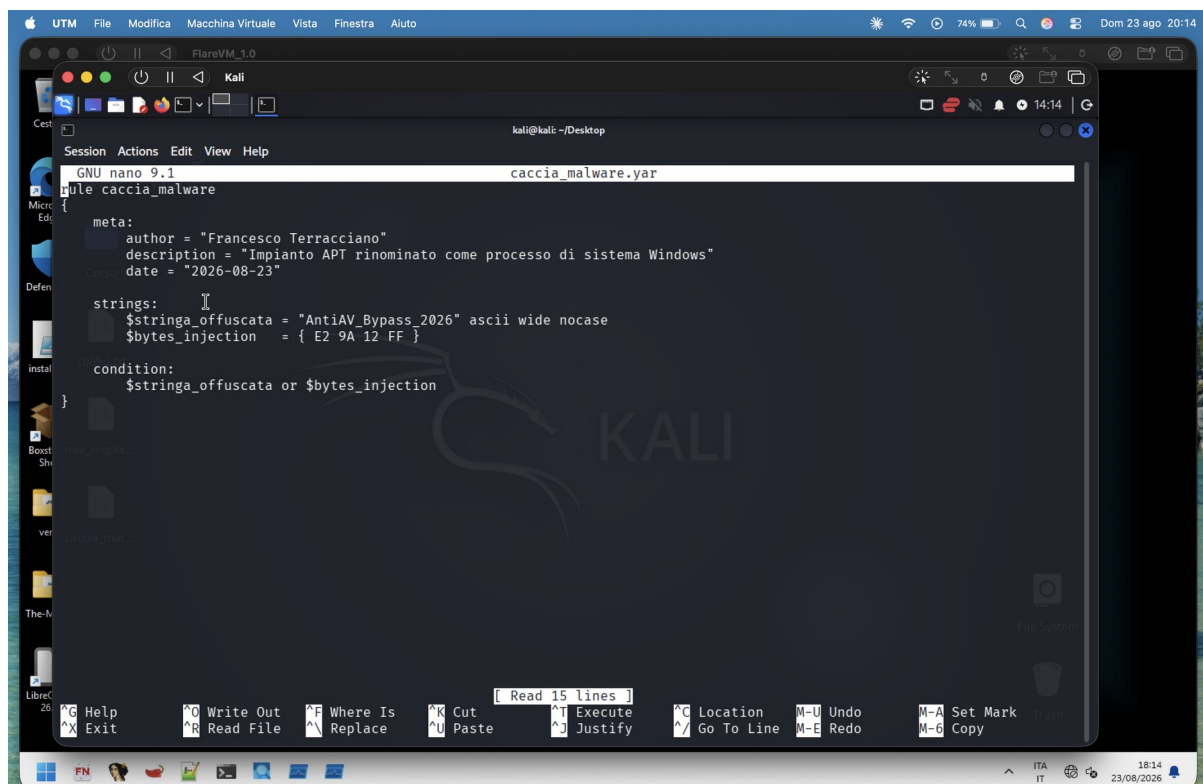


Fig. 3 — La regola caccia_malware.yar completa, con sezione meta, i due indicatori e la condizione in OR.

4.2 Nota metodologica: due errori corretti in corso d'opera

Il primo tentativo di scansione è fallito con l'errore "could not open file" perché la regola non era ancora stata salvata su disco. L'episodio è riportato perché evidenzia un aspetto non banale del comando: YARA distingue nettamente il file delle regole (primo argomento) dal bersaglio della scansione (secondo argomento), e l'errore si riferiva al primo.

```
(kali@kali)-[~/Desktop]
$ yara -r -s caccia_malware.yar /home/kali/Downloads/Flare_Incident_Drive.zip
error: could not open file: caccia_malware.yar
(kali@kali)-[~/Desktop]
```

Fig. 4 — Errore dovuto al file di regole non ancora creato.

Il secondo aspetto riguarda il bersaglio: nel medesimo tentativo la scansione era stata puntata direttamente sull'archivio .zip. YARA opera sui byte grezzi del file che riceve, quindi su un archivio compresso avrebbe letto soltanto dati compressi, all'interno dei quali le stringhe in chiaro non sono presenti. L'archivio è stato pertanto estratto prima della scansione.

4.3 Estrazione e scansione ricorsiva

```
cd ~/Downloads
unzip Flare_Incident_Drive.zip -d Directory_Sospetta
yara -r -s ~/Desktop/caccia_malware.yar ~/Downloads/Directory_Sospetta
```

L'opzione -r rende la scansione ricorsiva su tutte le sottocartelle, mentre -s stampa quale stringa ha prodotto il match e a quale offset: un'informazione decisiva per documentare il motivo della rilevazione e non soltanto il suo esito.

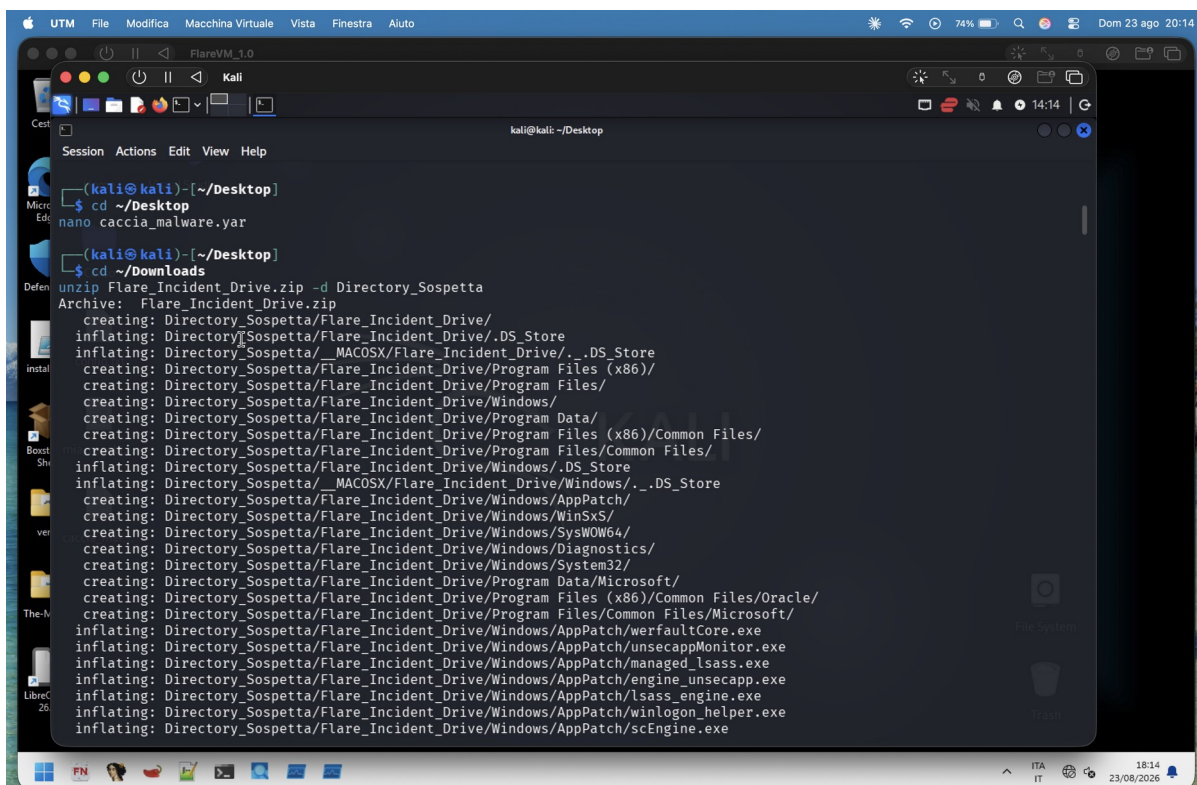


Fig. 5 — Estrazione dell'archivio: la struttura riproduce un filesystem Windows (Windows, System32, Program Files) con centinaia di eseguibili dai nomi verosimili.

```
(kali@kali)-[~/Downloads]
$ yara -r -s ~/Desktop/caccia_malware.yar ~/Downloads/Directory_Sospetta
caccia_malware /home/kali/Downloads/Directory_Sospetta/Flare_Incident_Drive/Windows/System32/drivers/updater_service.exe
0x1336:$stringa_offuscata: AntiAV_Bypass_2026
0x1b51:$bytes_injection: E2 9A 12 FF
```

Fig. 6 — Esito della scansione: un unico match su tutto l'albero, con entrambi gli indicatori rilevati.

4.4 Esito

La scansione ha prodotto un unico match su tutta la struttura analizzata:

```
caccia_malware
.../Flare_Incident_Drive/Windows/System32/drivers/updater_service.exe
0x1336:$stringa_offuscata: AntiAV_Bypass_2026
0x1b51:$bytes_injection: E2 9A 12 FF
```

Il file malevolo è updater_service.exe, collocato in Windows\System32\drivers. Sono stati rilevati entrambi gli indicatori sullo stesso file, agli offset 0x1336 e 0x1b51, senza alcun falso positivo: gli IoC forniti dal report si sono dimostrati estremamente selettivi.

5. Analisi del campione individuato

5.1 Identificazione

Attributo	Valore
Nome file	updater_service.exe
Percorso	Windows\System32\drivers\
SHA - 256	321fd5ac928c25b3b09c5b884cb7767b6f802cca945459342336194238b5459c
Tipo (file)	MS-DOS executable, MZ for MS-DOS
IoC rilevati	AntiAV_Bypass_2026 (0x1336) — E2 9A 12 FF (0x1b51)

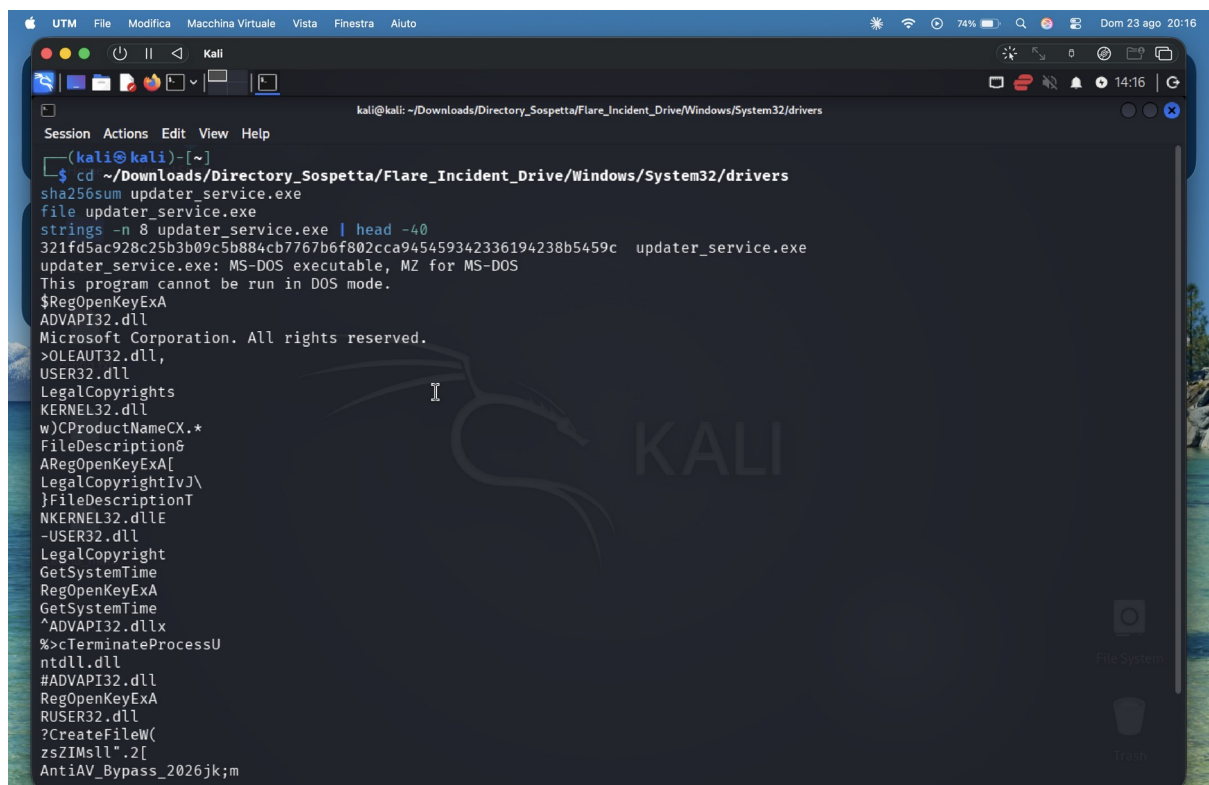


Fig. 7 — Triage del campione: hash SHA-256, riconoscimento del formato ed estrazione delle stringhe leggibili.

5.2 Indicatori comportamentali

Al di là degli IoC che hanno attivato la regola, il campione presenta tre anomalie significative.

- Collocazione incoerente. La cartella System32\drivers ospita driver in formato .sys, non eseguibili .exe. La presenza di un .exe in quella posizione è di per sé un indicatore, indipendentemente dal contenuto: sfrutta una directory di sistema considerata affidabile per apparire legittimo e per beneficiare di eventuali esclusioni antivirus.
- Masquerading tramite nome. Il nome updater_service.exe non corrisponde ad alcun processo reale di Windows: non imita un binario esistente, ma punta sulla genericità rassicurante della dicitura “updater service”.
- Metadati contraffatti. Le stringhe ProductName, FileDescription e LegalCopyright, insieme alla dicitura “Microsoft Corporation. All rights reserved.”, costituiscono metadati di versione falsificati che rafforzano la strategia di mimetizzazione.

5.3 API importate

Le funzioni individuate tra le stringhe delineano un profilo funzionale coerente con l'ipotesi di impianto malevolo:

API / libreria	Comportamento ipotizzabile
RegOpenKeyExA	Accesso al registro di sistema, tipico dei meccanismi di persistenza
CreateFileW	Operazioni di lettura e scrittura su file
TerminateProcess	Terminazione di processi, coerente con funzioni di evasione anti-AV
GetSystemTime	Logica temporale o rilevamento di ambienti sandbox
ntdll.dll	Accesso ad API di basso livello, potenziale aggiramento dei controlli in user-mode

Va segnalata una discrepanza rispetto al report di Threat Intelligence: pur essendo menzionata la memory injection, tra le stringhe estratte non compaiono le API tipicamente associate a quella tecnica (VirtualAllocEx, WriteProcessMemory, CreateRemoteThread).

5.4 Natura del campione

Il comando file restituisce “MS-DOS executable, MZ for MS-DOS” anziché la stringa attesa per un binario Windows a 64 bit: l'header MZ iniziale è presente, ma non è stato individuato un header PE valido all'offset indicato dal campo e_lfanew. Anche le stringhe estratte appaiono frammentate (NtKernel32.dll, ARegOpenKeyExA[, }FileDescriptionT), segno che sono state inserite tra byte casuali anziché provenire da una import table reale. Il campione è dunque un artefatto costruito a fini didattici: riproduce fedelmente gli indicatori di un impianto reale senza essere un eseguibile funzionante — circostanza che, per un'esercitazione di detection statica, non incide in alcun modo sulla validità del procedimento.

6. Conclusioni

L'esercitazione dimostra il limite strutturale della detection basata su hash. Il campione aveva modificato la propria firma crittografica e assunto un nome verosimile in una directory di sistema:

una ricerca per hash o per nome sarebbe fallita, e l'ispezione manuale di un filesystem con migliaia di file non sarebbe stata praticabile.

La detection per pattern ha invece funzionato perché opera su un piano diverso: non identifica il file, ma la tecnica. Stringhe interne e sequenze di byte legate al *modus operandi* di un gruppo APT sopravvivono alla ricompilazione, alla rinomina e alla modifica dell'hash, perché sono parte del funzionamento del codice e non della sua identità. È questa la ragione d'essere di YARA.

Sul piano metodologico, la scelta di condurre l'analisi da Kali Linux ha confermato che l'analisi statica non richiede un ambiente omogeneo al campione: si sono ottenuti risultati completi analizzando un binario Windows x86-64 da un sistema Linux arm64, con il vantaggio aggiuntivo di rendere impossibile per costruzione l'esecuzione accidentale del codice sotto esame.

Un possibile sviluppo consiste nel raffinare la regola per ridurre il rischio di falsi positivi in contesti reali — ad esempio vincolandola alla firma PE tramite il modulo *pe* di YARA, oppure aggiungendo una condizione sulla dimensione del file — e nel proseguire l'indagine sul campione individuato con l'analisi dinamica in ambiente FLARE VM opportunamente isolato.